EXPERIMENT – 4

AIM:

Develop a game-playing agent using minimax and alpha-beta pruning for Tic-Tac-Toe or a similar turn-based game.

OBJECTIVE:

To develop a game-playing agent for Tic-Tac-Toe using the Minimax algorithm with Alpha-Beta pruning, and to observe how pruning reduces the number of game-tree nodes evaluated while preserving the same optimal decision.

PROCEDURE:

1. Represent the Tic-Tac-Toe board as a 9-cell list and implement a function to detect a winner across all rows, columns and diagonals.
2. Implement the Minimax recursive evaluation function that assigns +10/-10 (adjusted by depth) for an AI win/loss and 0 for a draw at terminal states.
3. Extend Minimax with Alpha-Beta pruning: maintain alpha (best already guaranteed for the maximizer) and beta (best already guaranteed for the minimizer), and prune a branch when beta <= alpha.
4. Implement best_move() which tries every empty cell for the AI and picks the move with the highest minimax score.
5. Count nodes_evaluated on each AI turn to measure the effect of pruning as the game tree shrinks.
6. Simulate a full game: the AI (X) plays optimally via Minimax while the opponent (O) follows a fixed scripted sequence of moves.
7. Print the board after every move along with the number of nodes evaluated, and report the final result (win/loss/draw).

PROGRAM / CODE:

```
#================================================================== #
 PRACTICAL 4
 # Minimax with Alpha-Beta Pruning - Tic-Tac-Toe
 #================================================================== HUMAN = "O"
 AI = "X" EMPTY =
 " "
 defprint_board(board): rows = []
        for r in range(3):
              rows.append(" | ".join(board[r * 3:r * 3 + 3])) print(("\n" + "-" * 9 +
        "\n").join(rows))
 defwinner(board): lines = [
              (0, 1, 2), (3, 4, 5), (6, 7, 8),
              (0, 3, 6), (1, 4, 7), (2, 5, 8),
              (0, 4, 8), (2, 4, 6)
        ]
        for a, b, c in lines:
              if board[a] != EMPTY and board[a] == board[b] == board[c]: return board[a]
          return None
       def is_full(board):
        return EMPTY not in board
 nodes_evaluated = 0
 def minimax(board, depth, alpha, beta, maximizing): global
        nodes_evaluated
        nodes_evaluated += 1 win =
        winner(board) if win == AI:
              return 10 - depth
```

```
        if win == HUMAN: return
            depth - 10
        ifis_full(board): return 0
        if maximizing:
            best = float("-inf") for i in
            range(9):
                if board[i] == EMPTY: board[i]
                    = AI
                    score = minimax(board, depth + 1, alpha, beta, False) board[i] = EMPTY
                    best = max(best, score) alpha =
                    max(alpha, best) if beta <= alpha:
                        break
            return best else:
            best = float("inf") for i in
            range(9):
                if board[i] == EMPTY: board[i]
                    = HUMAN
                    score = minimax(board, depth + 1, alpha, beta, True) board[i] = EMPTY
                    best = min(best, score) beta =
                    min(beta, best) if beta <= alpha:
                        break
            return best def
best_move(board):
        best_score = float("-inf") move = None
        for i in range(9):
            if board[i] == EMPTY: board[i] = AI
                score = minimax(board, 0, float("-inf"), float("inf"), False) board[i] = EMPTY
                if  score  >  best_score:
                    best_score  =  score  move
                    = i
        return move
def scripted_human_move(board):
        """A fixed sequence representing a human/opponent playing sensibly.""" for i in [4, 0, 2, 6, 8, 1, 3, 5, 7]:
            if board[i] == EMPTY: return i
if __name___  == "__main__": board =
        [EMPTY] * 9 turn = AI
        move_no = 0
        print("=" * 55)
        print("          MINIMAX + ALPHA-BETA PRUNING - TIC-TAC-TOE")
        print("=" * 55) while
        True:
            move_no += 1
            print(f"\nMove {move_no} ({turn}'s turn)") if turn == AI:
                nodes_evaluated = 0 pos =
                best_move(board) board[pos]
                = AI
                print(f"AI plays position {pos} (nodes evaluated: {nodes_evaluated})") else:
                pos = scripted_human_move(board) board[pos] =
                HUMAN
                print(f"Opponent plays position {pos}") print_board(board)
            w = winner(board) if w:
                print(f"\nResult: {w} WINS!") break
            if is_full(board): print("\nResult: DRAW")
                break
            turn = HUMAN if turn == AI else AI
```

```
        print("=" * 55)
```

OUTPUT:

```
========================================================= MINIMAX +
      ALPHA-BETA PRUNING - TIC-TAC-TOE
 ========================================================
 Move 1 (X's turn)
 AI plays position 0                (nodes evaluated: 34202)
 X |       |
 ----------
   |       |
 ----------
   |       |
 Move 2 (O's turn) Opponent plays
 position 4
 X |       |
 ----------
   | O |
 ----------
   |       |
 Move 3 (X's turn)
 AI plays position 1                (nodes evaluated: 1630)
 X | X |
 ----------
   | O |
 ----------
   |       |
 Move 4 (O's turn) Opponent plays
 position 2
 X | X | O
 ----------
   | O |
 ----------
   |       |
 Move 5 (X's turn)
 AI plays position 6                (nodes evaluated: 97)
 X | X | O
 ----------
   | O |
 ----------
 X |       |
 Move 6 (O's turn) Opponent plays
 position 8
 X | X | O
 ----------
   | O |
 ----------
 X |       | O
 Move 7 (X's turn)
 AI plays position 3                (nodes evaluated: 10)
 X | X | O
 ----------
 X | O |
 ----------
 X |       | O Result: X
 WINS!
 ========================================================
```

RESULT:

The Minimax algorithm with Alpha-Beta pruning was successfully implemented for Tic-Tac-Toe. On the AI's first move (empty board, the largest possible search tree), Minimax evaluated 34,202 nodes even with pruning; as the game progressed and the board filled up, the number of nodes evaluated per move fell sharply (1,630 → 97 → 10) because there were fewer empty cells and more branches were pruned early once a guaranteed win/loss bound was found. Playing against a fixed opponent sequence, the AI played optimally at every turn and won the game, confirming that Alpha-Beta pruning returns exactly the same decision as plain Minimax while exploring a fraction of the game tree, which is

essential for making game-tree search practical even in a game as small as Tic-Tac-Toe.